

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

#	Scan ID 3		Date September 19, 2026
	Organization ensuro		Repository vaults
	Branch main		Commit Hash 3e856d7e...185534f1

Contracts in scope

contracts/interfaces/IExposeStorage.sol contracts/interfaces/IInvestStrategy.sol

contracts/strategies/AaveV3InvestStrategy.sol contracts/strategies/ChainlinkSwapAssetInvestStrategy.sol

contracts/strategies/CompoundV3InvestStrategy.sol contracts/strategies/ERC4626InvestStrategy.sol

contracts/strategies/IdleInvestStrategy.sol contracts/strategies/MerkleRewardsInvestStrategy.sol

contracts/strategies/MorphoVaultV2InvestStrategy.sol contracts/strategies/SwapAssetInvestStrategy.sol

contracts/strategies/SwapStableAaveV3InvestStrategy.sol

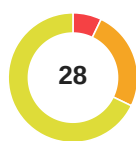
contracts/strategies/SwapStableInvestStrategy.sol contracts/AccessManagedMSV.sol

contracts/InvestStrategyClient.sol contracts/MSVBase.sol contracts/OutflowLimitedAMMSV.sol

Code Statistics

 Findings 28	 Contracts Scanned 16	 Lines of Code 2147
--	---	---

Findings Summary



Total Findings

 High Risk (2)	 Info (0)
 Medium Risk (7)	 Best Practices (0)
 Low Risk (19)	

Code Summary

The protocol provides a modular, multi-strategy ERC4626 vault architecture (Multi-Strategy Vault / MSV) designed to manage liquidity and optimize yields across decentralized finance protocols. It enables a single ERC4626 tokenized vault to allocate its underlying assets across up to 32 different investment strategies simultaneously, using customizable deposit and withdrawal queues. When users deposit funds, the vault fills strategies in the specified deposit priority order up to their deposit capacities. When withdrawals or redemptions occur, liquidity is sourced from strategies sequentially according to the withdrawal queue.

The architecture is structured around modular strategy components executing via `delegatecall` inside the vault's context:

- **Vault Core** (`MSVBase` , `AccessManagedMSV` , `OutflowLimitedAMMSV`): Handles ERC4626 share accounting, asset inflows/outflows, strategy indexing, rebalancing mechanisms, and queue configurations. `OutflowLimitedAMMSV` adds protective rate limiting on net outflows across rolling time windows to mitigate sudden liquidity drains. Permissions and strategy invocation access controls are integrated with OpenZeppelin's `IAccessManager` framework via `AccessManagedProxy`.
- **Pluggable Strategies** (`IInvestStrategy`): Standalone stateless contracts providing integration logic with external DeFi venues. Implementations include `AaveV3InvestStrategy` and `SwapStableAaveV3InvestStrategy` for Aave V3 lending pools, `CompoundV3InvestStrategy` for Compound V3 supply and reward harvesting, `ERC4626InvestStrategy` and `MorphoVaultV2InvestStrategy` for underlying ERC4626 and Morpho V2 vaults, `IdleInvestStrategy` for unallocated reserves, and `SwapAssetInvestStrategy` / `MerkleRewardsInvestStrategy` for automated token swaps and Merkle incentive claiming using Chainlink price feeds.
- **Storage Isolation and Strategy Execution** (`InvestStrategyClient` , `IExposeStorage`): Strategies operate directly on delegated vault storage partitioned via deterministic storage slots, allowing vaults to query strategy views and execute forwarded method calls safely.

Main Entry Points by Actor

Users and Liquidity Providers

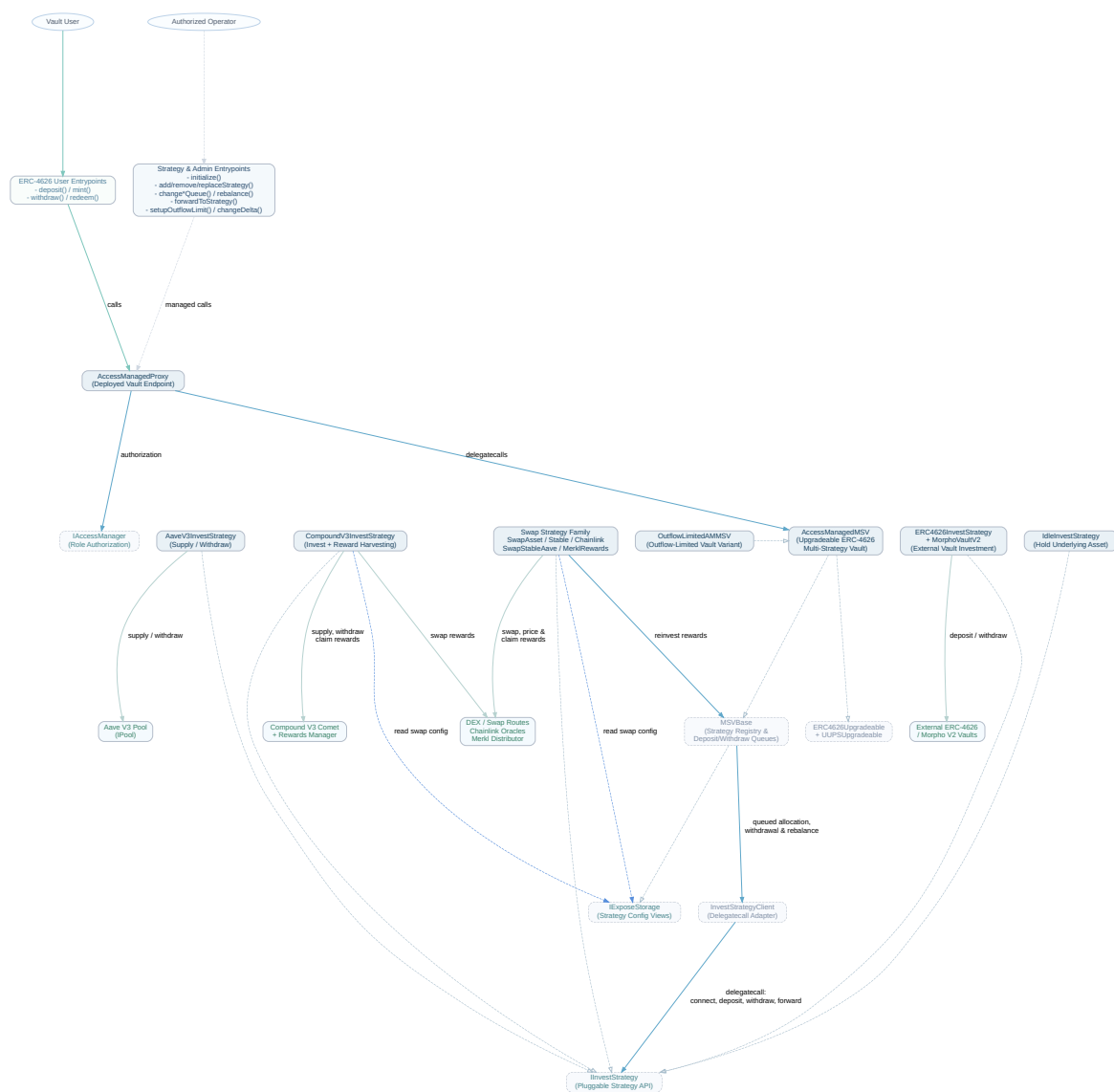
- `deposit(uint256 assets, address receiver)`: Deposits a specified amount of underlying assets into the vault in exchange for shares, routing funds through the deposit queue.
- `mint(uint256 shares, address receiver)`: Mints an exact number of vault shares by depositing the required amount of underlying assets into active strategies.
- `withdraw(uint256 assets, address receiver, address owner)`: Burns vault shares to withdraw a specific amount of underlying assets, pulling funds from strategies via the withdraw queue.
- `redeem(uint256 shares, address receiver, address owner)`: Burns an exact amount of shares to redeem underlying assets from strategies along the withdraw queue.

Vault Operators and Strategy Managers

- `initialize(string memory name_, string memory symbol_, IERC20 asset_, IInvestStrategy[] memory strategies)`: Initializes the upgradeable vault with target underlying assets, initial strategies, and default routing queues.
- `addStrategy(IInvestStrategy newStrategy, bytes memory initStrategyData)`: Connects a new investment strategy to the vault and appends it to the deposit and withdrawal queues.
- `removeStrategy(uint8 strategyIndex, bool force)`: Disconnects and removes an empty strategy from the vault and queue configurations.

- `replaceStrategy(uint8 strategyIndex, IInvestStrategy newStrategy, bytes memory initStrategyData, bytes memory withdrawStrategyData)`: Withdraws all assets from an existing strategy, disconnects it, and links a replacement strategy in place.
- `changeDepositQueue(uint8[] memory newDepositQueue_)`: Updates the priority order of strategies used when allocating deposited assets.
- `changeWithdrawQueue(uint8[] memory newWithdrawQueue_)`: Updates the priority order of strategies used when sourcing assets for withdrawals.
- `rebalance(uint8 strategyFromIdx, uint8 strategyToIdx, uint256 amount)`: Reallocates a specified asset amount directly from one strategy into another.
- `depositToStrategies(uint256 assets)`: Supplies unallocated or reinjected vault assets into connected strategies following the deposit queue.
- `forwardToStrategy(uint8 strategyIndex, uint8 method, bytes memory extraData)`: Executes permissioned external actions on a strategy, such as updating swap configurations, claiming rewards, or harvesting incentives.
- `setupOutflowLimit(uint256 slotSize, uint256 limit)`: Configures the time slot duration and maximum net outflow threshold allowed across consecutive time slots.
- `changeDelta(SlotIndex slot, int256 deltaChange)`: Manually adjusts the recorded net flow delta for a specific time slot to adjust or reset outflow limits.

Code Diagram



✦ 1 of 28 Findings

contracts/AccessManagedMSV.sol contracts/OutflowLimitedAMMSV.sol

Contract locks Ether without a withdraw function

• High Risk

The reported contract hierarchy includes payable Ether-handling behavior but no corresponding withdrawal entry point is shown. Ether sent to the deployed contract can therefore remain permanently held. The affected declarations are `AccessManagedMSV is MSVBase, UUPSUpgradeable, ERC4626Upgradeable` in `contracts/AccessManagedMSV.sol` and `OutflowLimitedAMMSV is AccessManagedMSV` in `contracts/OutflowLimitedAMMSV.sol`.

✦ 2 of 28 Findings

contracts/strategies/AaveV3InvestStrategy.sol

Reentrancy: State change after external call

• High Risk

The condition at `aave_.getReserveData(address(asset_)).aTokenAddress == address(0)` invokes the external Aave contract while evaluating the branch. The finding identifies state-changing logic after this external interaction, so a reentrant call could execute before the enclosing operation has completed and before its state effects are finalized.

✦ 3 of 28 Findings

contracts/MSVBase.sol

Maximum-size strategy configuration causes out-of-bounds panics in full-array traversals

• Medium Risk

MSVBase permits exactly 32 active strategies, but several loop conditions access a fixed-size array before checking `i < MAX_STRATEGIES`. With all 32 strategy slots occupied, `_totalAssets` processes indexes 0 through 31 and then evaluates `_strategies[32]`, reverting with an out-of-bounds panic. This breaks total-asset accounting and ERC4626 operations that depend on it. The queue-rewrite loop in `removeStrategy` similarly evaluates `_withdrawQueue[32]` after processing all 32 populated queue entries, so strategy removal reverts and cannot reduce the vault below the broken maximum-size configuration. Other affected traversals can also panic when execution reaches the end without an earlier return or completion. Bounds checks must be placed before every array access in the loop conditions.

Severity Note:

- Impact on live deposits/withdrawals assumes inheriting ERC4626 vaults call `_totalAssets` (and the max helpers) on those paths, which this abstract base is written to supply but does not wire itself.

✦ 4 of 28 Findings contracts/strategies/SwapStableAaveV3InvestStrategy.sol contracts/MSVBase.sol

Failed Aave resupply leaves investment tokens unaccounted, stranded, and detachable

• Medium Risk

`SwapStableAaveV3InvestStrategy.withdraw` intentionally withdraws the entire Aave position before satisfying even a partial vault withdrawal. It then attempts to resupply the unused `_investAsset` balance with `failSafe == true`. A failed resupply is swallowed, leaving the remaining investment tokens held directly by the vault.

```
function withdraw(uint256 assets) public virtual override onlyDelegCall {
    if (assets == 0) return;
    _aave.withdraw(address(_investAsset), type(uint256).max, address(this));
    super.withdraw(assets);
    _supply(_investAsset.balanceOf(address(this)), true);
}

function _supply(uint256 assets, bool failSafe) internal {
    if (assets == 0) return;
    _investAsset.approve(address(_aave), assets);
    if (failSafe) {
        try _aave.supply(address(_investAsset), assets, address(this), 0) {
            return;
        } catch {
            emit ResupplyFailed(assets);
        }
    } else _aave.supply(address(_investAsset), assets, address(this), 0);
}
```

The strategy's accounting only values aTokens. It omits both direct `_investAsset` inventory and any other residual balance left after the operation:

```
function totalAssets(address contract_) public view virtual override returns (uint256 assets) {
    return _convertAssets(IERC20Metadata(_reserveData().aTokenAddress).balanceOf(contract_), contract_);
}

function disconnect(bool force) external virtual override onlyDelegCall {
    IERC20Metadata aToken = IERC20Metadata(_reserveData().aTokenAddress);
    if (!force && aToken.balanceOf(address(this)) != 0) revert CannotDisconnectWithAssets();
}
```

After a failed resupply, the aToken balance can be zero while a substantial direct `_investAsset` balance remains in the vault. Therefore `totalAssets()` reports zero for this position and `disconnect(false)` permits detaching the strategy despite the residual tokens. `MSVBase` computes the vault's ERC4626 assets solely from strategy-reported values:

```
function _totalAssets() internal view returns (uint256 assets) {  
    for (uint256 i; address(_strategies[i]) != address(0) && i < MAX_STRATEGIES; ++i) {  
        assets += _strategies[i].totalAssets();  
    }  
}
```

This makes the residual investment tokens disappear from share accounting and from the strategy's reported withdrawal capacity. If the Aave reserve later accepts supply again, a subsequent deposit invokes `_supply(_investAsset.balanceOf(address(this)), false)`, which supplies both the new depositor's converted tokens and the previously unaccounted residual balance. Since share minting occurred while the residual balance was excluded from `totalAssets()`, the new depositor can receive shares at a materially deflated price and capture value belonging to existing shareholders. If the strategy is removed before recovery, the direct investment-token balance is left outside the accounting and routing system altogether.

Severity Note:

- Theft versus lock depends on a later successful supply (Aave unfreeze plus a deposit). The freeze/supply-failure itself is not attacker-controlled.
- Strategy removal without force is an operator action after `totalAssets` already reads zero; it worsens stranding but is not required for the accounting bug.

✦ 5 of 28 Findings

contracts/MSVBase.sol contracts/strategies/IdleInvestStrategy.sol

Distinct strategies can double-count the same vault-owned position

• Medium Risk

MSVBase prevents reusing the exact same strategy address, but it does not ensure that separately deployed strategy contracts value disjoint positions. This is material because strategies operate over positions owned by the vault address, not positions owned by an individual strategy address.

```
for (uint256 j; j < i; ++j) {
    if (strategies[i] == strategies[j]) revert DuplicatedStrategy(strategies[i]);
}

...

function _totalAssets() internal view returns (uint256 assets) {
    for (uint256 i; address(_strategies[i]) != address(0) && i < MAX_STRATEGIES; ++i) {
        assets += _strategies[i].totalAssets();
    }
}
```

For example, two distinct deployments of IdleInvestStrategy pass the address-duplication check and both pass `checkAsset`, but each reports the entire liquid asset balance of the same vault:

```
function totalAssets(address contract_) public view virtual override returns (uint256
assets) {
    return _asset.balanceOf(contract_);
}
```

Consequently, if the vault contains 100 accounting assets and two IdleInvestStrategy instances are active, `_totalAssets()` reports 200 assets even though only 100 exist. The same overlap is possible with two separately deployed Aave strategies configured for the same reserve, because both inspect the vault's identical aToken balance.

The inflated accounting directly corrupts ERC4626 share pricing. In the two-idle-strategy example, if 100 shares exist against 100 actual assets but reported AUM is 200, a holder redeeming 50 shares can be quoted approximately 100 assets. `_withdrawFromStrategies` treats Idle withdrawals as fulfilled because their `withdraw` functions are no-ops, after which the ERC4626 layer transfers the actual liquid balance. The first redeemer can therefore extract assets backing other outstanding shares, leaving remaining shareholders with unbacked shares or failed redemptions.

Severity Note:

- The insolvency path assumes an inheriting ERC4626 uses `_totalAssets` for share conversion and transfers the vault's liquid asset on withdraw; that child is not in the provided files.
- `addStrategy` and `replaceStrategy` are public virtual in MSVBase; this rating assumes inheritors restrict them to a trusted strategy-management role.

6 of 28
Findingscontracts/MSVBase.sol contracts/AccessManagedMSV.sol
contracts/strategies/SwapAssetInvestStrategy.sol**Direct underlying balances omitted from totalAssets enable share-price dilution**

• Medium Risk

`AccessManagedMSV.totalAssets()` delegates to `MSVBase._totalAssets()`, which sums only `totalAssets()` reported by registered strategies. It never adds the vault's direct `_asset()` balance. Although an `IdleInvestStrategy` may report the vault's raw accounting-token balance, neither initialization nor strategy management requires such a strategy to be installed, nor otherwise ensures that direct assets are represented exactly once.

Consequently, underlying tokens held directly by the vault are absent from ERC4626 accounting whenever no installed strategy includes that balance in its reported assets. Such a balance may result from unsolicited transfers, residual assets left by management flows, or ordinary `SwapAssetInvestStrategy` operation. In particular, when `SwapAssetInvestStrategy.withdraw` handles a full withdrawal, it swaps its entire investment-token balance through `exactInput` when the requested amount is at least its conservative, slippage-adjusted valuation. The swap output can exceed the requested withdrawal amount. After the ERC4626 withdrawal transfers the requested amount to the receiver, this excess remains as direct underlying tokens in the vault. The swap strategy's `totalAssets()` values only the investment-token balance and therefore does not report that surplus.

Deposits, mints, conversions, withdrawals, and redemptions use this understated `totalAssets()` value. A depositor can acquire shares while an existing direct balance `D` is omitted, receiving shares priced against only strategy-reported assets `A`. If `D` is later deposited into a reporting strategy, or otherwise becomes reported during strategy allocation or replacement, the vault's accounted assets increase without corresponding share issuance. For existing supply `S` and an attacker deposit `X`, the attacker receives approximately $X * S / A$ shares rather than the lower amount based on $A + D$; after recognition, their claim is approximately $X + X * D / (A + X)$, aside from ERC4626 rounding. This transfers part of the pre-existing direct balance's value from existing holders to the depositor. An attacker can front-run an observable allocation or strategy transition that will recognize the omitted balance, then redeem after the accounting transition. More generally, omitted balances cause AUM and share pricing to diverge from the assets economically owned by the vault, allowing value redistribution independent of the shares that owned those balances.

Severity Note:

- Captured amount equals a fraction of the omitted idle balance, not vault TVL; size tracks swap leftover versus maxSlippage or external transfers.
- Recognition is typically an ordinary allocation or strategy-management call, not a proven permissionless one-transaction drain.

7 of 28
Findings

```
contracts/MSVBase.sol  contracts/strategies/AaveV3InvestStrategy.sol
contracts/strategies/CompoundV3InvestStrategy.sol
contracts/strategies/SwapStableAaveV3InvestStrategy.sol
contracts/strategies/MorphoVaultV2InvestStrategy.sol
```

Strategies overstate executable liquidity and deposit capacity, blocking MSV queue operations

• Medium Risk

MSVBase treats each strategy's `maxWithdraw` and `maxDeposit` as executable bounds. It aggregates these values when exposing vault limits and uses them to select `toWithdraw` or `toDeposit` for queue processing. The resulting delegatecalls are made with errors not ignored, so a revert from the selected strategy aborts the operation before MSVBase can proceed to later queue entries.

Several strategies report accounting claims or `type(uint256).max` rather than the capacity their external protocol can actually execute. `AaveV3InvestStrategy.maxWithdraw` returns the vault's full aToken balance without limiting it to Aave's immediately available underlying liquidity. Its `maxDeposit` returns unlimited capacity whenever the reserve is active, unpaused, and unfrozen, while ignoring the Aave supply cap.

`CompoundV3InvestStrategy.maxWithdraw` likewise returns the full Comet balance when withdrawals are not paused, even though Compound's available base-token liquidity can be lower than that receipt-token claim.

`SwapStableAaveV3InvestStrategy.maxWithdraw` reports the accounting conversion of its full Aave position and its `maxDeposit` also ignores the Aave supply cap. Its withdrawal behavior further requires Aave to redeem the entire Aave position for every nonzero withdrawal, because it calls Aave's `withdraw` with `type(uint256).max` before completing the requested withdrawal and attempting to resupply any remainder. Thus, even a smaller requested withdrawal can revert when Aave cannot redeem the entire position.

`MorphoVaultV2InvestStrategy` returns `totalAssets` as its withdrawal capacity and unlimited deposit capacity, without respecting the connected ERC4626 vault's `maxWithdraw(address(vault))` and `maxDeposit(address(vault))` limits. Its reported accounting value can therefore exceed the amount that the Morpho vault will actually allow the strategy to withdraw or deposit.

Consequently, a withdrawal or redemption can be accepted as within the vault's reported aggregate withdrawal limit, but fail when the first selected Aave, Compound, SwapStable Aave, or Morpho strategy cannot fulfill its advertised amount. The revert prevents MSVBase from using a later queued strategy that has sufficient liquidity. Likewise, deposits or mints can fail at an Aave supply cap or Morpho deposit limit despite a later queued strategy having capacity, because the first strategy advertises unlimited deposit capacity. An authorized rebalance out of an illiquid Aave or Compound strategy can also pass MSVBase's `maxWithdraw` check and then revert during the external withdrawal.

✦ 8 of 28 Findings

contracts/strategies/CompoundV3InvestStrategy.sol

CompoundV3InvestStrategy._harvestRewards executes external call to depositToStrategies instead of internal execution

• Medium Risk

In `CompoundV3InvestStrategy._harvestRewards`, the proceeds from swapping claimed Compound V3 rewards are reinjected by invoking `MSVBase(address(this)).depositToStrategies(reinvestAmount);`:

```
function _harvestRewards(uint256 price) internal {
    (address reward, , ) = _rewardsManager.rewardConfig(address(_cToken));
    if (reward == address(0)) return;
    _rewardsManager.claim(address(_cToken), address(this), true);

    SwapLibrary.SwapConfig memory swapConfig = abi.decode(
        StorageSlot.getBytesSlot(storageSlot).value,
        (SwapLibrary.SwapConfig)
    );

    uint256 earned = IERC20(reward).balanceOf(address(this));
    uint256 reinvestAmount = swapConfig.exactInput(reward, _baseToken, earned, price);

    MSVBase(address(this)).depositToStrategies(reinvestAmount);
    emit RewardsClaimed(reward, earned, reinvestAmount);
}
```

Because the strategy runs in the vault's context via `delegatecall`, `address(this)` is the `AccessManagedProxy`. Calling `MSVBase(address(this)).depositToStrategies(...)` performs an external call back to the proxy, where `msg.sender` is the proxy itself (`address(this)`). In the `AccessManagedMSV` architecture, calls to non-public methods routed through `AccessManagedProxy` require authorization in the OpenZeppelin `AccessManager`. Unless the proxy contract itself has explicitly been granted permissions in the `AccessManager` to call `depositToStrategies` on itself, this external call will revert with `AMPUtls.AccessManagedUnauthorized`.

In contrast, `MerkleRewardsInvestStrategy` resolves this by executing

```
ERC1967Utils.getImplementation().functionDelegateCall(abi.encodeCall(MSVBase.depositToStrategies, amount))
```

Consequently, reward harvesting in `CompoundV3InvestStrategy` reverts during execution.

Severity Note:

- AccessManagedProxy and AccessManager self-call behavior are not in the supplied strategy file; severity assumes that architecture denies unauthenticated proxy self-calls.
- Harm is inability to complete harvest, not a demonstrated execute path that moves principal.

contracts/strategies/CompoundV3InvestStrategy.sol

9 of 28 Findings

contracts/strategies/MerkleRewardsInvestStrategy.sol

contracts/strategies/SwapAssetInvestStrategy.sol

Unaccounted accrued rewards can be captured by just-in-time depositors

• Medium Risk

Compound rewards owed to the vault are omitted from `CompoundV3InvestStrategy.totalAssets()` until an authorized harvest claims and swaps them. The strategy accounts only for the Compound principal balance:

```
function totalAssets(address contract_) public view virtual override returns (uint256 assets) {
    return _cToken.balanceOf(contract_);
}
```

The reward is recognized only after the harvest transaction has claimed it and reinvested its conversion output:

```
_rewardsManager.claim(address(_cToken), address(this), true);

uint256 earned = IERC20(reward).balanceOf(address(this));
uint256 reinvestAmount = swapConfig.exactInput(reward, _baseToken, earned, price);

MSVBase(address(this)).depositToStrategies(reinvestAmount);
```

The same accounting discontinuity exists for unclaimed Merkle rewards. `MerkleRewardsInvestStrategy` inherits balance-based valuation from `SwapAssetInvestStrategy`, which values only reward tokens already held by the vault:

```
function totalAssets(address contract_) public view virtual override returns (uint256 assets) {
    return _convertAssets(_investAsset.balanceOf(contract_), contract_);
}
```

Merkle rewards are absent from that balance until a permitted call supplies a claim proof and executes the distributor claim:

```
(amounts[0], proofs[0]) = abi.decode(params, (uint256, bytes32[]));
users[0] = address(this);
tokens[0] = investAsset(address(this));
distributor.claim(users, tokens, amounts, proofs);
```

A user can observe a pending authorized harvest or `claimAndSwapRewards` transaction, deposit immediately before it, and receive shares priced without the already-accrued reward entitlement. Once the subsequent transaction claims and reinvests the rewards, the vault's reported assets increase and the new shares receive the same proportional claim on those rewards as the shares that funded the underlying position while rewards accrued. The depositor can then redeem their shares, transferring part of the pre-existing accrued rewards from

existing shareholders. The gain is not limited to rewards generated during the attacker's holding period; it includes the reward inventory accrued before the deposit.

Severity Note:

- Economic size of the extraction equals unclaimed rewards at harvest time; the code explicitly recommends frequent claims, so materiality is an operating assumption rather than a constant.

✦ 10 of 28 Findings

contracts/OutflowLimitedAMMSV.sol

OutflowLimitedAMMSV does not constrain maxWithdraw and maxRedeem to configured outflow limit

• Low Risk

`OutflowLimitedAMMSV` restricts withdrawals and redemptions by checking that net asset outflows across the current and preceding time slots do not exceed `$.limit`:

```
function _withdraw(
    address caller,
    address receiver,
    address owner,
    uint256 assets,
    uint256 shares
) internal virtual override {
    LOMStorage storage $ = _getLOMStorage();
    if ($slotSize != 0) {
        SlotIndex slot = _slotIndex();

        SlotIndex prevSlot = SlotIndex.wrap(SlotIndex.unwrap(slot) - 1);
        int256 deltaLastTwoSlots = -int256(assets) + $.assetsDelta[slot] +
$.assetsDelta[prevSlot];
        if (deltaLastTwoSlots < 0 && uint256(-deltaLastTwoSlots) > $.limit)
            revert LimitReached(deltaLastTwoSlots, $.limit);

        $.assetsDelta[slot] -= assets.toInt256();
    }
    super._withdraw(caller, receiver, owner, assets, shares);
}
```

However, `OutflowLimitedAMMSV` does not override `maxWithdraw(address owner)` or `maxRedeem(address owner)`. As a result, both functions return values based only on user balance and strategy liquidity, ignoring the outflow capacity remaining under `$.limit`.

According to the ERC-4626 standard, `maxWithdraw` and `maxRedeem` must return the maximum amount of assets/shares that can be withdrawn or redeemed without reverting. When the remaining outflow allowance is smaller than the owner's balance, calling `withdraw(maxWithdraw(owner))` or `redeem(maxRedeem(owner))` reverts with `LimitReached`, violating ERC-4626 specifications and causing failures for automated integrators.

✦ 11 of 28 Findings

contracts/InvestStrategyClient.sol contracts/MSVBase.sol

strategyChange sweeps IdleInvestStrategy reserve assets into the replacement strategy

• Low Risk

When replacing an investment strategy, `MSVBase.replaceStrategy` delegates the transition to `InvestStrategyClient.strategyChange`:

```
function strategyChange(
    IInvestStrategy oldStrategy,
    IInvestStrategy newStrategy,
    bytes memory newStrategyInitData,
    IERC20Metadata asset,
    bool force
) internal {
    checkAsset(newStrategy, address(asset));
    dcWithdraw(oldStrategy, oldStrategy.totalAssets(address(this)), force);
    dcDisconnect(oldStrategy, force);
    dcConnect(newStrategy, newStrategyInitData);
    // Deposits all the funds, in case something was gifted to the vault.
    dcDeposit(newStrategy, asset.balanceOf(address(this)), force);
    emit StrategyChanged(oldStrategy, newStrategy);
}
```

`strategyChange` retrieves the funds managed by `oldStrategy` into `address(this)` and deposits `asset.balanceOf(address(this))` into `newStrategy`.

When `IdleInvestStrategy` is also installed in the vault, its accounting relies directly on `asset.balanceOf(contract_)` held in the vault context. Executing `strategyChange` on any other strategy causes `dcDeposit` to deposit the entirety of `asset.balanceOf(address(this))`—which includes all assets belonging to `IdleInvestStrategy`—into `newStrategy`. Consequently, `IdleInvestStrategy` is drained to zero, stripping the vault of its liquid reserve.

Severity Note:

- `IdleInvestStrategy` source was not in the provided contracts; severity assumes it accounts vault-held `asset.balanceOf` as described.
- `replaceStrategy` is public virtual on `MSVBase`; analysis assumes inheriting vaults keep it as a trusted operator function, consistent with the finding. Missing access control would be a separate, much larger issue.

✦ 12 of 28 Findings

contracts/strategies/IdleInvestStrategy.sol

Replacing a funded IdleInvestStrategy with force=false always reverts

• Low Risk

InvestStrategyClient.strategyChange withdraws the old strategy's reported assets before disconnecting it. For IdleInvestStrategy, withdraw is a no-op because the assets already reside at the vault, so the vault's asset balance remains nonzero. The subsequent disconnect(false) treats that unchanged vault balance as assets still requiring withdrawal and reverts with CannotDisconnectWithAssets. Consequently, MSVBase.replaceStrategy cannot perform the documented fail-closed replacement of a funded Idle strategy; operators must instead use force mode, which suppresses other migration failures, or perform a separate multi-step rebalance. This defect concerns replacement; removeStrategy(false) independently and intentionally rejects a strategy reporting nonzero assets before disconnect is reached.

Severity Note:

- InvestStrategyClient.strategyChange and MSVBase.replaceStrategy were not in the supplied sources; the withdraw-then-disconnect(false) sequence is taken from the finding text.
- force=true remains a working disconnect for IdleInvestStrategy because idle funds already sit on the vault; the residual issue is only loss of fail-closed checks on the rest of the swap.

✦ 13 of 28 Findings

contracts/MSVBase.sol contracts/strategies/SwapAssetInvestStrategy.sol

Normal removal of swap-based strategies reverts after the strategy is deregistered

• Low Risk

MSVBase removes a strategy from `_strategies` before invoking its `disconnect(false)` hook.

```
for (; i < MAX_STRATEGIES && _strategies[i] != IInvestStrategy(address(0)); ++i) {
    _strategies[i - 1] = _strategies[i];
}
_strategies[i - 1] = IInvestStrategy(address(0));

...

strategy.dcDisconnect(force);
```

The inherited SwapAssetInvestStrategy disconnection check calls `totalAssets(address(this))` when `force` is false:

```
function disconnect(bool force) external virtual override onlyDelegCall {
    if (!force && totalAssets(address(this)) != 0) revert CannotDisconnectWithAssets();
}
```

Its total-assets calculation reads the strategy's swap configuration from the vault through `getBytesSlot`:

```
function _getSwapConfig(address contract_) internal view returns (SwapLibrary.SwapConfig memory) {
    bytes memory swapConfigAsBytes = IExposeStorage(contract_).getBytesSlot(storageSlot);
    return abi.decode(swapConfigAsBytes, (SwapLibrary.SwapConfig));
}
```

However, `MSVBase.getBytesSlot` only exposes a storage slot for a strategy that is still present in `_strategies`. Since the target strategy has already been shifted out, the call reverts with `OnlyStrategyStorageExposed` before the zero-assets condition can be evaluated. Thus, a zero-balance SwapAssetInvestStrategy, SwapStableInvestStrategy, ChainlinkSwapAssetInvestStrategy, or MerklRewardsInvestStrategy cannot be removed through the normal `force == false` path. This blocks normal strategy lifecycle operations and forces use of the emergency-style removal mode even where no residual position exists.

✦ 14 of 28 Findings

contracts/MSVBase.sol

Costly operations inside loop

• Low Risk

Five reported loop instances in `contracts/MSVBase.sol` iterate with `for (; i < newWithdrawQueue_.length; ++i)`. The analyzer associates these loops with `SSTORE` work, so gas usage grows with the number of queued withdrawal entries and may become expensive for large `newWithdrawQueue_` arrays.

✦ 15 of 28 Findings

contracts/MSVBase.sol

`delegatecall` in loop

• Low Risk

Two reported loop instances in `contracts/MSVBase.sol` use the loop `for (uint256 i; left != 0 && _depositQueue[i] != 0 && i < MAX_STRATEGIES; ++i)`. Delegate-call work performed for each qualifying deposit-queue entry can consume gas proportional to the number of iterations, and delegate-called code executes in the caller's storage context.

✦ 16 of 28 Findings

contracts/MSVBase.sol

Loop Contains `require` / `revert`

• Low Risk

Eight reported instances in `contracts/MSVBase.sol` contain the loop `for (; i < newWithdrawQueue_.length; ++i)` and are associated with `require` or `revert` paths inside the loop body. A failure while processing one withdrawal-queue element can consequently revert the entire transaction, including work performed for earlier elements in the same iteration sequence.

✦ 17 of 28 Findings

contracts/MSVBase.sol

Uninitialized Local Variable

• Low Risk

Eight reported instances in `contracts/MSVBase.sol` declare the loop counter inline as `for (uint256 i; i < MAX_STRATEGIES && _strategies[i] != IInvestStrategy(address(0)); ++i)`. The counter relies on Solidity's default initialization rather than an explicit initializer, which can obscure the intended starting index when reviewing strategy traversal logic.

contracts/AccessManagedMSV.sol contracts/OutflowLimitedAMMSV.sol

18 of 28 Findings

contracts/strategies/MerkleRewardsInvestStrategy.sol

contracts/strategies/MorphoVaultV2InvestStrategy.sol

Unused Import

• Low Risk

The analyzer identifies four redundant imports: `IERC4626` from `contracts/AccessManagedMSV.sol`, `ERC4626Upgradeable` from `contracts/OutflowLimitedAMMSV.sol`, and `IInvestStrategy` from each of `contracts/strategies/MerkleRewardsInvestStrategy.sol` and `contracts/strategies/MorphoVaultV2InvestStrategy.sol`. The imported symbols are not referenced by the analyzed source, increasing source-level dependency noise without contributing to compilation.

19 of 28 Findings

contracts/strategies/SwapAssetInvestStrategy.sol

contracts/strategies/SwapStableAaveV3InvestStrategy.sol

Contract has TODO Comments

• Low Risk

The analyzer found TODO comments associated with the contract declarations `SwapAssetInvestStrategy is IInvestStrategy` and `SwapStableAaveV3InvestStrategy is SwapStableInvestStrategy`. These markers indicate unfinished or intentionally deferred work in the implementation area and should be reviewed against the intended production behavior.

20 of 28 Findings

contracts/OutflowLimitedAMMSV.sol

contracts/strategies/SwapAssetInvestStrategy.sol

Literal Instead of Constant

• Low Risk

The analyzer reports two instances in `contracts/OutflowLimitedAMMSV.sol` of the literal arithmetic expression `SlotIndex.wrap((slotSize << 128) + block.timestamp / slotSize)`, and three instances in `contracts/strategies/SwapAssetInvestStrategy.sol` of `10 ** (18 - token.decimals())`. These repeated numeric literals are embedded directly in calculations, which makes their meaning and consistency harder to enforce across the affected code paths.

✦ 21 of 28 Findings

contracts/AccessManagedMSV.sol contracts/strategies/IdleInvestStrategy.sol

Empty Block

• Low Risk

The analyzer reports an empty upgrade authorization hook in `contracts/AccessManagedMSV.sol`:
`function _authorizeUpgrade(address newImpl) internal view override {}`. It also reports two instances of the empty `deposit` override in `contracts/strategies/IdleInvestStrategy.sol`:
`function deposit(uint256 assets) public virtual override onlyDelegCall {}`. These bodies contain no statements beyond their modifiers and override declarations, making their behavior dependent entirely on inherited or modifier logic.

✦ 22 of 28 Findings

contracts/InvestStrategyClient.sol

Internal Function Used Only Once

• Low Risk

Five reported instances identify
`function checkAsset(IInvestStrategy strategy, address asset) internal view` in
`contracts/InvestStrategyClient.sol` as an internal helper used only once. The helper introduces an additional internal call boundary for the asset validation logic without evidence of reuse elsewhere in the analyzer output.

✦ 23 of 28
Findingscontracts/strategies/CompoundV3InvestStrategy.sol
contracts/strategies/ERC4626InvestStrategy.sol**Public Function Not Used Internally**

• Low Risk

`getSwapConfig(address contract_) public view returns (SwapLibrary.SwapConfig memory)` in
`CompoundV3InvestStrategy.sol` and `investVault() public view returns (IERC4626)` in
`ERC4626InvestStrategy.sol` are reported as public functions with no internal callers. Their public visibility exposes them through the contract ABI even though the analyzer found no need for internal dispatch.

✦ 24 of 28 Findings

📁 contracts/OutflowLimitedAMMSV.sol

Unused State Variable

• Low Risk

`contracts/OutflowLimitedAMMSV.sol` declares`bytes32 private constant STORAGE_LOCATION = 0xa2ada5d673dba5eecea7c7503ee87e29913d0d36ae093e950d632f7b868`

which the analyzer reports as unused. No reference to this constant was identified in the analyzed Solidity code, so it currently does not affect the contract's runtime behavior.

✦ 25 of 28
Findings`contracts/AccessManagedMSV.sol` `contracts/MSVBase.sol``contracts/OutflowLimitedAMMSV.sol` `contracts/interfaces/IExposeStorage.sol``contracts/interfaces/IInvestStrategy.sol` `contracts/strategies/AaveV3InvestStrategy.sol``contracts/strategies/ChainlinkSwapAssetInvestStrategy.sol``contracts/strategies/CompoundV3InvestStrategy.sol``contracts/strategies/ERC4626InvestStrategy.sol` `contracts/strategies/IdleInvestStrategy.sol``contracts/strategies/MerkleRewardsInvestStrategy.sol``contracts/strategies/MorphoVaultV2InvestStrategy.sol``contracts/strategies/SwapAssetInvestStrategy.sol``contracts/strategies/SwapStableAaveV3InvestStrategy.sol``contracts/strategies/SwapStableInvestStrategy.sol`**Unspecific Solidity Pragma**

• Low Risk

All fifteen listed files declare `pragma solidity ^0.8.0;`. The caret permits compilation with any compatible compiler version from 0.8.0 onward, so builds can differ in compiler behavior, generated bytecode, and bug fixes unless the project build configuration independently pins the compiler version.

26 of 28
Findings

contracts/InvestStrategyClient.sol contracts/MSVBase.sol
contracts/strategies/AaveV3InvestStrategy.sol
contracts/strategies/CompoundV3InvestStrategy.sol
contracts/strategies/ERC4626InvestStrategy.sol
contracts/strategies/MerkleRewardsInvestStrategy.sol
contracts/strategies/SwapAssetInvestStrategy.sol
contracts/strategies/SwapStableAaveV3InvestStrategy.sol

Unchecked Return

• Low Risk

The analyzer reports ignored return values at all of the following locations: six

`dcDeposit(newStrategy, asset.balanceOf(address(this)), force);` calls in `contracts/InvestStrategyClient.sol`; four `strategyTo.dcDeposit(amount, false);` calls in `contracts/MSVBase.sol`; `IERC20(_asset).approve(address(_aave), assets);` in `AaveV3InvestStrategy.sol`; `IERC20(_baseToken).approve(address(_cToken), assets);` in `CompoundV3InvestStrategy.sol`; three `_vault.deposit(assets, address(this));` calls in `ERC4626InvestStrategy.sol`; `ERC1967Utils.getImplementation().functionDelegateCall(abi.encodeCall(MSVBase.depositToStrategies, amount))` in `MerkleRewardsInvestStrategy.sol`; three `_getSwapConfigSelf().exactInput(address(_asset), address(_investAsset), assets, investAssetPrice());` calls in `SwapAssetInvestStrategy.sol`; and two `_investAsset.approve(address(_aave), assets);` calls in `SwapStableAaveV3InvestStrategy.sol`. Any success or result value returned by these calls is discarded, so the surrounding code does not explicitly validate it.

27 of 28
Findings

contracts/strategies/AaveV3InvestStrategy.sol
contracts/strategies/CompoundV3InvestStrategy.sol
contracts/strategies/ERC4626InvestStrategy.sol
contracts/strategies/SwapStableAaveV3InvestStrategy.sol

Unsafe ERC20 Operation

• Low Risk

The affected strategies invoke ERC20 `approve` directly: `IERC20(_asset).approve(address(_aave), assets)` in `AaveV3InvestStrategy.sol`, `IERC20(_baseToken).approve(address(_cToken), assets)` in `CompoundV3InvestStrategy.sol`, `_asset.approve(address(_vault), assets)` in `ERC4626InvestStrategy.sol`, and `_investAsset.approve(address(_aave), assets)` in `SwapStableAaveV3InvestStrategy.sol`. Direct calls may not handle tokens that return no boolean, return false instead of reverting, or impose allowance-transition requirements, and the reported return values are not checked.



28 of 28

Findings

contracts/strategies/AaveV3InvestStrategy.sol

contracts/strategies/CompoundV3InvestStrategy.sol

contracts/strategies/ERC4626InvestStrategy.sol contracts/strategies/IdleInvestStrategy.sol

contracts/strategies/SwapAssetInvestStrategy.sol

Empty `require()` / `revert()` Statement

• Low Risk

The reported failure paths use bare `revert()` without a reason string or custom error. The instances are in `AaveV3InvestStrategy.sol` (`revert()`), `CompoundV3InvestStrategy.sol` (`else revert()`), `ERC4626InvestStrategy.sol` (`revert()`), `IdleInvestStrategy.sol` (`revert()`), and `SwapAssetInvestStrategy.sol` (`else revert()`). Callers receive no diagnostic reason identifying which validation or external operation failed.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.